

NL-to-SQL Lakehouse

An engineering research report on DuckDB star schema + semantic layer +
text-to-SQL

Sai Veda

Independent Project Research

2024

Contents

1. Abstract	3
2. Introduction	3
2.1 Research Problem	3
2.2 Research Questions and Hypotheses	3
3. Technical Background Review	4
3.1 Design Foundations	4
3.2 Supporting Examples	4
4. Research Method	4
5. Data Description	5
6. Analysis	5
6.1 Benchmark and Validation Results	5
6.2 Test Result Screenshots	5
6.3 Validation Checklist	6
7. Discussion	6
8. Conclusion	6
9. Future Work	7
10. References	7

1. Abstract

This brief captures a natural-language analytics system built on a real lakehouse pattern: a large partitioned warehouse, a semantic layer, and a deterministic text-to-SQL compiler with safety checks. The project demonstrates that natural-language analytics becomes much more credible when it is backed by a semantic layer and SQL validation. The result is a system that answers business questions in English while still producing auditable SQL. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

Keywords: DuckDB star schema + semantic layer + text-to-SQL, implementation evidence, benchmark results, project validation

2. Introduction

Business users ask in plain language, but trusted reporting still depends on correct joins, guarded metrics, and a stable schema contract. The goal was to bridge those worlds without hiding behind an opaque model call. A DuckDB lakehouse over a partitioned-Parquet star schema, with a semantic layer and a deterministic, offline text-to-SQL engine. Ask a business question in English - "top 5 product categories by revenue", "which region has the lowest revenue?", "monthly revenue in 2023" - and the system links schema entities, compiles validated SQL through the semantic layer, and executes it out-of-core against 50M fact rows.

2.1 Research Problem

Business users ask in plain language, but trusted reporting still depends on correct joins, guarded metrics, and a stable schema contract. The goal was to bridge those worlds without hiding behind an opaque model call.

2.2 Research Questions and Hypotheses

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: 50M-row lakehouse; 100% NL->SQL execution accuracy.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 36 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Technical Background Review

3.1 Design Foundations

- 1 The semantic layer is what makes NL-to-SQL trustworthy rather than merely executable. The semantic layer and SQL guardrails are the load-bearing parts that keep natural-language analytics trustworthy.
- 2 DuckDB was chosen because it keeps the warehouse story local, fast, and transparent.
- 3 Validation is a product feature: unsafe or schema-breaking queries should fail early and clearly.

3.2 Supporting Examples

- Fact and dimension generation into partitioned Parquet.
- DuckDB execution layer over the lakehouse.
- Entity linking, metric mapping, and validated SQL generation.

4. Research Method

The project was reviewed as an engineering artefact using repository documentation, architecture notes, benchmark evidence, automated tests, and generated screenshots. Modeled the dataset as a star schema so business questions map cleanly to measures and dimensions. Added a metric layer to keep NL questions aligned with curated business logic. Converted linked entities and intents into validated SQL that can be inspected before execution. Measured heavy queries at 50M rows to show that the analytic path remains credible at scale.

5. Data Description

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 36 automated tests, 0 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

6. Analysis

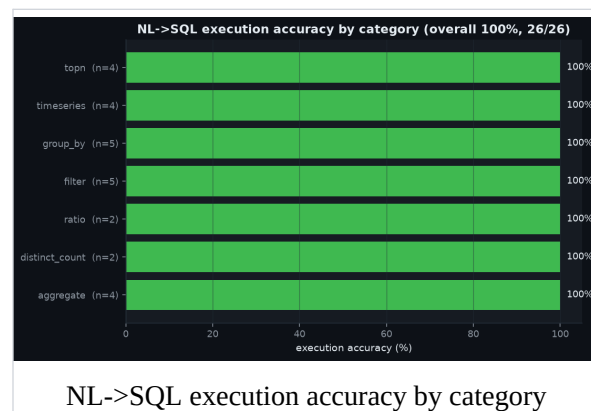
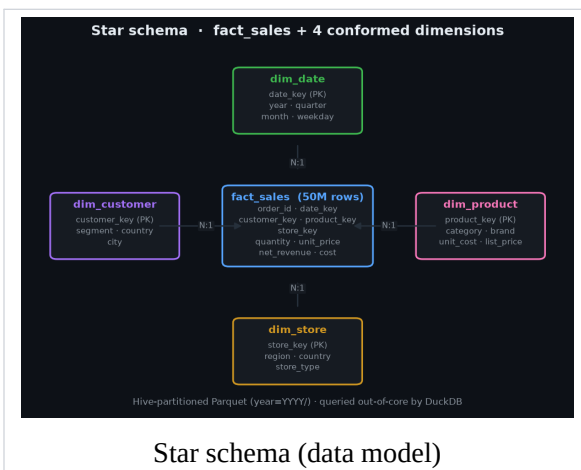
6.1 Benchmark and Validation Results

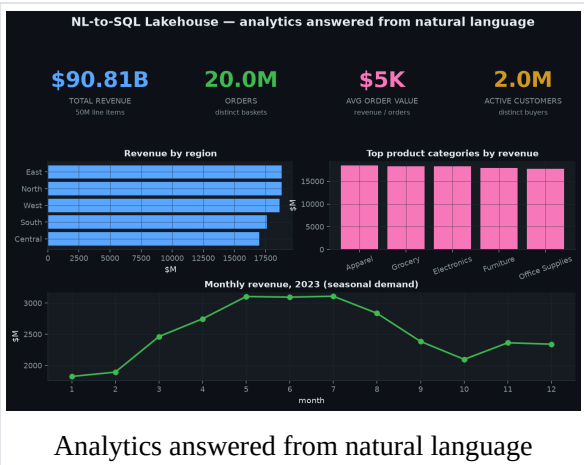
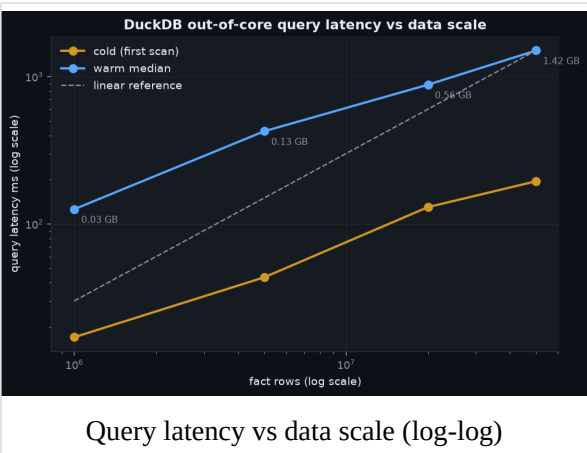
The headline repository result is reported as: 50M-row lakehouse; 100% NL->SQL execution accuracy. The table below reproduces benchmark evidence included in the repository where available.

Execution accuracy matters because every natural-language answer is backed by runnable SQL, not narrative text alone. The lakehouse benchmarks show that large-table scans and joins remain practical on one machine. The screenshots reinforce that trust comes from visible SQL and visible schema linkage, not from hidden prompt logic.

6.2 Test Result Screenshots

The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.





6.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	36 tests committed in repository validation flow.
Benchmark results	Limited	No benchmark markdown tables were present; discussion relies on screenshots and h
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

7. Discussion

The project demonstrates that natural-language analytics becomes much more credible when it is backed by a semantic layer and SQL validation. The result is a system that answers business questions in English while still producing auditable SQL. Execution accuracy matters because every natural-language answer is backed by runnable SQL, not narrative text alone. The lakehouse benchmarks show that large-table scans and joins remain practical on one machine. The screenshots reinforce that trust comes from visible SQL and visible schema linkage, not from hidden prompt logic. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

8. Conclusion

This project document concludes that NL-to-SQL Lakehouse is best understood as a complete technical implementation supported by repository-native evidence. This brief

captures a natural-language analytics system built on a real lakehouse pattern: a large partitioned warehouse, a semantic layer, and a deterministic text-to-SQL compiler with safety checks.

9. Future Work

- Support more complex cohort, funnel, and time-series comparison queries.
- Add approval workflows for generated SQL in regulated or finance-facing contexts.
- Connect the semantic layer to upstream catalog metadata and freshness indicators.

10. References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/accuracy_by_category.csv - benchmark or result artefact used in the analysis section.
4. benchmarks/generation_report.json - benchmark or result artefact used in the analysis section.
5. benchmarks/scaling_results.csv - benchmark or result artefact used in the analysis section.
6. benchmarks/suite_results.csv - benchmark or result artefact used in the analysis section.
7. tests/ - automated validation suite used to support implementation claims.
8. assets/accuracy_by_category.png - generated screenshot evidence included in the report.